PFM-STM32G474 — Serial Command Reference

Every command below is read from `Core/Src/commands.c` / `Core/Src/cmd_parser.c` / `Core/Inc/supply_config.h` directly — this is a reference, not a re-description from memory.

Link: USART2, 9600 8N1, lines terminated `\r\n`. Commands are case-sensitive, uppercase, space-separated (`cmd_parser.c`'s own convention comment). An unrecognized verb gets `ERR 1 Unknown command`.

Responses: either `OK ...\r\n` (some commands add data after `OK`; several diagnostic dumps send multiple lines before a final `OK`), or `ERR <code> <message>\r\n`. See the error code table below.

Build variants: firmware compiles for exactly one `SUPPLY_TYPE` (`supply_config.h`) — **HVPS**, **LIMITER**, or **DEBUG**. Several commands are only available on some of these; each entry below says which. Two independent feature toggles can also remove whole command groups at compile time: `FEEDBACK_FEATURE_ENABLED` (VTGT/FBLOG?/ADC?) and `FIBER_ENABLE_FEATURE_ENABLED` (not a serial command — a hardware trigger pin, not covered here).

Contents

1. State machine and mode
2. Error codes
3. Identification / system
4. State machine control
5. Mode select
6. CONSTANT mode setpoints — `SET`
7. Query commands
8. SWEEP mode table builder
9. Raw register-table upload — `TBL_*`
10. Gate driver / fault status
11. FEEDBACK mode
12. Diagnostics
13. Per-build availability matrix
14. Bounds and defaults by supply type

State machine and mode

Two independent pieces of state gate most commands:

State (`SM_State_t`, `state_machine.h`) — `IDLE → ARMED → FIRING`, with `FAULT` reachable from anywhere:

```
IDLE  --ARM-->   ARMED  --FIRE-->  FIRING
IDLE  <-DISARM-  ARMED  <-STOP---  FIRING (early abort)
(any) --fault--> FAULT  --CLEARFAULTS-->  IDLE
```

- `ARM` fails with `ERR 2` if `Fault_IsLockedOut()`, `ERR 4` if no runnable mode/params exist yet, `ERR 6` if not currently Idle.
- A shot ends on table exhaustion, a fault, or `STOP` — there is no pulse-length watchdog timer separate from the built table.

Mode (`CmdMode_t`, `commands.c` — separate from `SM_State_t`) — `NONE → CONSTANT | SWEEP | FEEDBACK`, settable only while Idle via `MODE`. Controls which setpoint command family is meaningful; `ARM` always rebuilds the table from whatever mode/setpoints currently exist.

Error codes

Code	Meaning	Typical cause
1	Unknown command	Typo, or a command unavailable on this build's compile-time feature set (absent from the table entirely, not just rejected)
2	Busy / locked out	A shot is active, the controller is fault-locked, or the specific command requires Idle
4	Not runnable / wrong mode	`ARM` with no mode/params configured; a FEEDBACK-only command sent outside `MODE FEEDBACK`
5	Usage / syntax error	Missing or malformed arguments
6	Transition rejected	Bad state-machine transition, or `MODE` sent while not Idle
7	Frequency out of range	Outside this build's `ACTIVE_FREQ_MIN/MAX_HZ`
8	Duty (or uploaded value) out of range	Outside `ACTIVE_DUTY_MIN/MAX_PCT`, or a `TBL_DATA` value outside the uploaded array's allowed range
9	Pulse length out of range	`SET PULSE` parsed to zero
11	Table/segment build failed	Track length mismatch, segment exceeds `PFM_TABLE_SIZE`, or `*_INIT` never called before `*_SEG`
12	Not available on this build	Blocked by `SUPPLY_TYPE` gating (`SUPPLY_ALLOW_*` in `supply_config.h`)
13	List full	`VTGT_ADD`/`VTGT_INIT` beyond `PFM_FB_SETPOINTS_MAX` (10)

Identification / system

`*IDN`

No args. `OK PFM-STM32G4 BOOT Test 2 <SUPPLY_TYPE_NAME> <FW_VERSION_STRING>\r\n`, e.g. `OK PFM-STM32G4 BOOT Test 2 SUPPLY_LIMITER v0.1`.

`*RST`

No args. Resets the **command layer**: mode back to `NONE`, all setpoints back to build defaults, tracks cleared, any raw upload discarded. Rejects with `ERR 2` while Firing. Does **not** force an `SM_State_t` transition — if Armed, you stay Armed. Does **not** touch phase enable/disable state (`SET ENABLE`/`DISABLE`) or `Fault_IsLockedOut()` (only `CLEARFAULTS` clears lockout).

State machine control

Command	Args	Response	Notes
`ARM`	none	`OK`	Rebuilds the table (`PFM_BuildTable()`, or `PFM_BuildFeedbackTable()` in `MODE FEEDBACK`, or verbatim if a `TBL_BUILD` upload exists) every time, from current setpoints. `ERR 11` on build failure, `ERR 2`/`4`/`6` per state machine.
`DISARM`	none	`OK`	Armed → Idle. `ERR 6` if not Armed.
`FIRE`	none	`OK`	Armed → Firing; starts real switching immediately. `ERR 6` if not Armed.
`STOP`	none	`OK`	Firing → Armed (early abort). `ERR 6` if not Firing.
`CLEARFAULTS`	none	`OK`	Fault → Idle. `ERR 2` if the underlying fault condition is still active (lockout not clearable yet), `ERR 6` if not in Fault.
`STATUS`	none	`OK STATE <s> MODE <m> LOCKOUT <0|1> IDX <i>/<n>\r\n`	`IDX` is `-/-` unless currently Firing.

Mode select

`MODE CONSTANT\|SWEEP\|FEEDBACK`

Only while Idle (`ERR 6` otherwise). Discards any raw `TBL_*` upload. `FEEDBACK` only exists if `FEEDBACK_FEATURE_ENABLED`. `OK` on success, `ERR 5` on a bad/missing sub-argument.

CONSTANT mode setpoints — `SET`

`SET <SUB> <value>` (or `SET ENABLE|DISABLE <phase>`, no numeric value). **All of these are available in either `MODE CONSTANT` or `MODE SWEEP`** — per project decision they are the shared startup/fallback values `PFM_BuildTable()` uses for any axis without a built SWEEP track, not gated to CONSTANT mode specifically.

Sub-command	Args	Availability	Notes
`SET FREQ <hz>`	integer Hz	`SUPPLY_ALLOW_CONST_FREQ` (all three builds)	Bounds: [`ACTIVE_FREQ_MIN_HZ`, `ACTIVE_FREQ_MAX_HZ`] → `ERR 7` outside range
`SET DUTY <pct>`	integer %	always	Writes **U/V/W together**. Bounds: [`ACTIVE_DUTY_MIN_PCT`, `ACTIVE_DUTY_MAX_PCT`] → `ERR 8`
`SET DUTY_U|DUTY_V|DUTY_W <pct>`	integer %	`SUPPLY_ALLOW_CONST_PER_PHASE_DUTY` (HVPS: no — true 3-phase, single duty only)	Individual phase only
`SET ENABLE|DISABLE U|V|W`	phase letter, not numeric	always	Gates that phase's HRTIM **output** only (its timer counter keeps running). **Durable** — survives `*RST` and `MODE` changes; only another `SET ENABLE/DISABLE` changes it. See `ENABLED?`.
`SET PULSE <ms>`	decimal, one fractional digit (e.g. `3.1`) — extra precision is **truncated**, not rounded	always	Tenths-of-a-millisecond resolution. `ERR 9` if it parses to zero.
`SET FB_CTRL_N <n>`	integer ≥ 1	`FEEDBACK_FEATURE_ENABLED`	Periods between feedback control-loop updates (default 10).

Query commands

Command	Response	Availability
`FREQ?`	`OK <hz>`	`SUPPLY_ALLOW_CONST_FREQ`
`DUTY?`	`OK <pct>`	always
`DUTY_U?` / `DUTY_V?` / `DUTY_W?`	`OK <pct>`	`SUPPLY_ALLOW_CONST_PER_PHASE_DUTY`

Command	Response	Availability
`ENABLED?`	`OK U=<0|1> V=<0|1> W=<0|1>`	always
`PULSE?`	`OK <ms>.<tenth>`	always
`MODE?`	`OK NONE|CONSTANT|SWEEP|FEEDBACK`	always
`TRACK? [FREQ|DUTY|DUTY_U|DUTY_V|DUTY_W]`	multi-line segment-log dump, `OK` last	With no argument, dumps every track this build actually has. Each entry: `INIT value=N` or `SEG init=A next=B steps=S dwell=D`, capped at `PFM_TRACK_LOG_MAX` = 32 logged entries per track (the built track itself can be much longer — this is a debug log, not the full expansion).

`FREQ?`/`DUTY?`/etc. report the **effective** value: whatever was last `SET`, or the build default if never set — there's no separate "not yet set" state.

SWEEP mode table builder

Requires `MODE SWEEP` and `SM_STATE_IDLE` (`ERR 6`/`ERR 2` otherwise). Builds a linear-interpolated segment track in RAM; nothing is applied to hardware until `ARM`. `<steps>`/`<dwell>` must each be ≥ 1.

A segment runs from the **current** value (the last `*_INIT` or the previous segment's target) to `<next>`, over `<steps>` interpolated points, each held for `<dwell>` table entries — so one segment contributes `steps × dwell` entries. The final step always lands exactly on `<next>` (rounding is absorbed there, not spread across the ramp).

Command	Args	Availability
`FREQ_INIT <hz>`	starting Hz	`SUPPLY_ALLOW_FREQ_CMDS` (HVPS, DEBUG — **not** LIMITER, fixed carrier)
`FREQ_SEG <next_hz> <steps> <dwell>`	as above	same
`DUTY_INIT <pct>`	starting %	Only when `SUPPLY_PER_PHASE_DUTY == 0` **and** `SUPPLY_ALLOW_DUTY_CMDS` — none of the three current builds actually satisfy both (HVPS has no duty track at all; LIMITER/DEBUG are per-phase)
`DUTY_SEG <next_pct> <steps> <dwell>`	as above	same
`DUTY_INIT_U|_V|_W <pct>`	starting %	`SUPPLY_PER_PHASE_DUTY` (LIMITER, DEBUG)
`DUTY_SEG_U|_V|_W <next_pct> <steps> <dwell>`	as above	same
`BUILD_END`	none	always; optional early preview of what `ARM` would build — `OK <n> entries` or `ERR 11`

`*_SEG` before the matching `*_INIT` (or a segment that would exceed `PFM_TABLE_SIZE` = 5000 total entries): `ERR 11`.

HVPS specifically: no duty track exists at all — `MODE SWEEP` always uses the current `SET DUTY` value for every table entry; `DUTY_INIT`/`DUTY_SEG` (and `_U`/`_V`/`_W` variants) are unavailable (`ERR 12`). **LIMITER:** no frequency track — carrier is fixed per shot via `SET FREQ`.

Raw register-table upload — TBL_*

Bypasses the freq/duty math entirely: streams raw HRTIM register counts (PER, CMP_U, CMP_V, CMP_W) directly into the table. Requires SM_STATE_IDLE. Which arrays are accepted depends on SUPPLY_TYPE:

Array	LIMITER	HVPS	DEBUG
PER	rejected (ERR 12 — carrier fixed)	required	allowed
CMP_U/CMP_V/CMP_W	required (per-phase)	rejected (ERR 12 — duty is shared, use SET DUTY)	allowed

Flow:

```
TBL_BEGIN <array>          -> OK <array> expect <PFM_TABLE_SIZE> values
TBL_DATA v1 v2 ...         -> OK +<n> (<filled>/<PFM_TABLE_SIZE>)   (repeat)
TBL_END                    -> OK   (checks the array got exactly PFM_TABLE_SIZE values)
... repeat TBL_BEGIN/DATA/END for each required array ...
TBL_BUILD                  -> OK built <n> entries; ARM to fire
```

- TBL_DATA values must be 0–65535 (ERR 5 otherwise).
- TBL_END on a short/long array: ERR 11.
- TBL_BUILD before every required array is complete: ERR 11 (names the offending array). Out-of-range value within an otherwise complete array: ERR 8.
- TBL_BUILD success marks the table runnable **verbatim** — the next ARM uses it as-is rather than rebuilding from SET/*_INIT/*_SEG state. Any subsequent MODE command discards it.
- TBL? reports staging state without side effects: OK TBL need <n> values/array, built=yes|no, then one line per relevant array — <name> <filled>/<n> complete|-.

Gate driver / fault status

Command	Response	Notes
GDS?	OK 0b<12 bits>	Raw GateDriverStatus_01..12 (PE0–PE11) snapshot, direct register read — bypasses the debounced/cached fault_pins.c state by design, so it shows the live instantaneous pin state. MSB-first: bit 11 (PE11 / status 12) on the left, bit 0 (PE0 / status 1) on the right.
FAULT?	one <name> FAULT|clean line per fault pin, OK last	Every fault pin this build has: all 12 GateDriverStatus_01..12, plus SparkGap on HVPS only. Debounced/cached state (unlike GDS?).
SPARKGAP?	OK FAULT|clean	HVPS only — ERR 12 on LIMITER/DEBUG. Single-pin convenience query, same underlying table as FAULT?.

FEEDBACK mode

Everything in this section only exists when FEEDBACK_FEATURE_ENABLED (all three current builds have it on by default). Requires MODE FEEDBACK for the setpoint-mutating commands; the queries are ungated.

Command	Args	Notes
VTGT_INIT <durationUs> <mV>	duration ≥ 1 µs, 0 ≤ mV ≤ ADC_VREF_MV (2900)	Clears the setpoint list and adds this as the first entry. Only while Idle (ERR 2 otherwise) and in MODE FEEDBACK (ERR 4 otherwise).
VTGT_ADD <durationUs> <mV>	same	Appends one more setpoint. Up to PFM_FB_SETPOINTS_MAX = 10 total; ERR 13 once full.

Command	Args	Notes
`VTGT?`	`OK <n> setpoints`, then `[i] <durUs> us <mV> mV` per entry, `OK` last	Read-only, always safe — no gating.
`FBLOG?`	`OK <n> entries`, then `[i] meas=<c> tgt=<c> per=<p> cmp=<c>` per entry, `OK` last	Post-shot control-loop log. Rejected with `ERR 2` only while Firing (log is being written mid-shot, and UART is disabled during a shot anyway).
`ADC?`	`OK A1=<raw> (<mV> mV) A2=... ... A8=...`	One reading per analog input `AnalogIn_01`–`08` (raw 12-bit count + derived mV). A channel that failed to convert shows `A<n>=FAIL` instead. DMA-driven, non-polling (see recent ADC1/ADC2 scan-mode work) — not just PA0/VREFINT as in an earlier revision.

Diagnostics

Human-readable multi-line dumps via `debug.c`, printed directly (not the usual `OK`-prefixed single-line framing on the dump body itself), each followed by a trailing bare `OK\r\n` so a host script can detect completion.

Command	Content
`REGS`	Live HRTIM register dump (`Debug_PrintHrtimLiveRegisters()`).
`PHASE`	Phase-relationship analysis across U/V/W (`Debug_PrintPhaseAnalysis()`).
`CONFIG`	Current PFM/supply configuration dump (`Debug_PrintPfmConfig()`).

Per-build availability matrix

Command family	HVPS	LIMITER	DEBUG
`SET FREQ` / `FREQ?`	✓	✓	✓
`SET DUTY_U/V/W` / `DUTY_U/V/W?`	× (true 3-phase, one duty)	✓	✓
`FREQ_INIT` / `FREQ_SEG`	✓	× (fixed carrier)	✓
`DUTY_INIT` / `DUTY_SEG` (shared)	×	×	×
`DUTY_INIT_U/V/W` / `DUTY_SEG_U/V/W`	× (no duty track at all — uses `SET DUTY`)	✓	✓
`TBL_BEGIN PER`	✓	×	✓
`TBL_BEGIN CMP_U/V/W`	×	✓	✓
`SPARKGAP?`	✓	×	×
`SET ENABLE/DISABLE`, `ENABLED?`	✓	✓	✓

(`DUTY_INIT`/`DUTY_SEG` — the shared, non-per-phase track — has no build where it's actually reachable today: it requires `SUPPLY_PER_PHASE_DUTY == 0` and `SUPPLY_ALLOW_DUTY_CMDS == 1` simultaneously, which none of HVPS/LIMITER/DEBUG currently satisfy. It exists for a hypothetical future shared-duty-track supply type.)

Bounds and defaults by supply type

	HVPS	LIMITER	DEBUG
Freq range	10,000–30,000 Hz	75,000–125,000 Hz	0–500,000 Hz
Freq default	20,000 Hz	100,000 Hz	0 Hz

	HVPS	LIMITER	DEBUG
Duty range	40–60 %	5–95 %	0–100 %
Duty default	50 %	50 %	50 %
Pulse default	5 ms	5 ms	5 ms

`PFM_TABLE_SIZE` = 5000 entries, fixed across all builds — the ceiling for total SWEEP-track length, uploaded `TBL_*` array length, and CONSTANT-mode pulse-length × frequency.